

User guide for GADfit version 2.1

September 2026

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Terms of use	3
2	Method	3
2.1	Modified Levenberg-Marquardt	4
2.2	Global nonlinear optimization	8
2.3	Automatic differentiation	8
2.3.1	Forward mode	9
2.3.2	Reverse mode	12
2.3.3	Implementation	14
2.3.4	AD with numerical integration	15
3	Building and linking	18
3.1	Building process	19
3.1.1	C++ prerequisites	20
3.1.2	Fortran prerequisites	20
3.1.3	CMake configuration variables	21
3.2	Runtime linking	21
3.2.1	C++	21
3.2.2	Fortran	22
3.3	Notes on using GADfit in other CMake projects	23
4	Usage	24
4.1	Parallelism	24
4.1.1	C++	24
4.1.2	Fortran	24
4.2	Example input	25
4.2.1	C++	25
4.2.2	Fortran	25
4.3	Defining the fitting function	26
4.3.1	C++	26
4.3.2	Fortran	26

4.4	Fitting procedure	27
4.4.1	C++	27
4.4.2	Fortran	28
4.5	User interface	30
4.5.1	C++	30
4.5.2	Fortran	34
4.6	Internals	43
4.6.1	Fortran	43
5	Developer notes	47
5.1	C++	47
5.1.1	Unit tests	47
5.1.2	Code linters	47
5.1.3	Code coverage	47
5.1.4	Coding rules	48
5.2	Fortran	48
5.2.1	Coding rules	48
5.3	Version control	49
6	Summary	50
7	Troubleshooting	50

1 Introduction

GADfit is an implementation of **g**lobal nonlinear curve fitting based on **a**utomatic differentiation (AD). Global fitting refers to fitting many data sets simultaneously with some parameters shared among the data sets but not necessarily finding the global minimum in parameter space. The optimization procedure is based on a modified Levenberg-Marquardt algorithm which is well-suited for difficult large scale problems with a strong interdependence of parameters. The Jacobian and other quantities requiring differentiation are evaluated using AD instead of finite differences which ensures that the derivatives are always calculated with the same precision as function evaluation. The cost of computing the derivatives is *independent* of the number of fitting parameters and is a small constant of function evaluation resulting in very efficient code. The method can be used for fitting functions of high complexity which, in the present implementation, include nonlinear combinations of elementary and special functions, single or double integrals, and any control flow statements allowed by the programming language.

GADfit is currently hosted by GitHub.

1.1 Contributors

Go to <https://github.com/raullaasner/gadfit/graphs/contributors> to see a list of contributors.

1.2 Terms of use

Licensed under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

2 Method

GADfit is based on the Levenberg-Marquardt (LM) algorithm [1], which is a standard technique for solving nonlinear least squares problems. A least squares problem refers to minimizing the sum of the squares of the differences between the data and a curve constructed from a parameterized function. If the model curve (the fitting function) has a nonlinear dependence on the fitting parameters, it is a nonlinear least squares problem.

Given a set of n data points (x_i, y_i) , where x_i are the independent variables, and a model function $f(x, \beta_1, \dots, \beta_k)$, which depends on x and k additional parameters (the fitting parameters), the sum of squares is defined as

$$\chi^2 = \sum_{i=1}^n \left[\frac{y_i - f(x_i, \boldsymbol{\beta})}{\sigma_i} \right]^2, \quad (1)$$

where σ_i are the data point uncertainties (standard deviations). As with all nonlinear optimization algorithms, Levenberg-Marquardt is an iterative procedure. One starts with an initial guess for the parameter vector $\boldsymbol{\beta}$ and finds an increment $\boldsymbol{\delta}$ such that $\boldsymbol{\beta} \rightarrow \boldsymbol{\beta} + \boldsymbol{\delta}$ leads to the lowering of χ^2 . This is repeated until some convergence criterion is satisfied. Following the original LM method, the increment vector (the step) is given by

$$[\mathbf{J}^T \mathbf{J} + \lambda \text{diag}(\mathbf{J}^T \mathbf{J})] \boldsymbol{\delta} = \mathbf{J}^T \mathbf{R}, \quad (2)$$

where¹

$$J_{ij} = \frac{1}{\sigma_i} \frac{\partial f(x_i, \boldsymbol{\beta})}{\partial \beta_j} \quad (3)$$

defines the Jacobian,

$$R_i = \frac{y_i - f(x_i, \boldsymbol{\beta})}{\sigma_i} \quad (4)$$

defines the residual vector, and where the damping parameter λ allows the nature of the algorithm to adaptively change between the steepest descent and the Gauss-Newton method. If χ^2 is far from its minimum, λ should have a large value, bringing the algorithm closer to steepest descent. On the other hand,

¹Usually the uncertainties σ_i are not included in the definition.

close to the minimum, Gauss-Newton is a better choice. A simple strategy for updating λ would be as follows: if for the current iteration χ^2 is smaller than for the previous iteration, accept the step and decrease λ by a factor of 10. Otherwise increase λ , solve Eq. (2), and recalculate χ^2 with the new step. If χ^2 still increases, stop the procedure, otherwise proceed with the next iteration. If the procedure stops after the first iteration, start again with a larger initial value for λ . A sufficiently large λ always exists that leads to a decrease of χ^2 [1].

2.1 Modified Levenberg-Marquardt

What was described above is the classical LM algorithm, which is close to the default regime of GADfit. In this section, we discuss several interesting modifications to the algorithm (see [2, 3, 4] and references therein), which aim to improve the convergence speed and the sensitivity to the initial guess.

The damping matrix. Originally, the rationale for choosing the damping matrix as $\mathbf{D}^T \mathbf{D} \equiv \text{diag}(\mathbf{J}^T \mathbf{J})$ in Eq. (2) was to make the algorithm invariant to the rescaling of parameters. This is important when the parameters have very different magnitudes and the algorithm has to follow a narrow canyon in the parameter space in search of the minimum of χ^2 . However, this poses a problem when the algorithm is in a region of space where the model is insensitive to the parameters. When this happens, the diagonal entries of both $\mathbf{J}^T \mathbf{J}$ and the damping matrix $\mathbf{D}^T \mathbf{D}$ become small for some parameters, which leads to uncontrollably large steps. There is a chance that some parameters are incorrectly pushed to infinite values, a phenomenon known as parameter evaporation. A compromise for both retaining the invariance to rescaling and guarding against parameter evaporation is to set the diagonal entries of $\mathbf{D}^T \mathbf{D}$ to the largest diagonal entries of $\mathbf{J}^T \mathbf{J}$ yet encountered. This approach is efficient provided that the initial guess does not lie on a plateau of the χ^2 surface in the parameter space. If this condition is not satisfied then, as a simple and robust solution, the user can specify the minimum values for the diagonal of $\mathbf{D}^T \mathbf{D}$.

Optimal path towards the minimum. Following a geometric interpretation of nonlinear optimization, it has been suggested that a geodesic in the parameter space is the most natural path for the algorithm to follow. When the algorithm is navigating a narrow curved canyon then, instead of taking simple steps along the gradient of χ^2 , it would be more efficient to take the curvature of the canyon into account and move along parabolic trajectories. This is especially beneficial for functions like the Rosenbrock function. To second order, the step is then given by

$$\begin{aligned}\boldsymbol{\delta} &= \boldsymbol{\delta}_1 + \boldsymbol{\delta}_2, \\ \boldsymbol{\delta}_1 &= [\mathbf{J}^T \mathbf{J} + \lambda \mathbf{D}^T \mathbf{D}]^{-1} \mathbf{J}^T \mathbf{R}, \\ \boldsymbol{\delta}_2 &= -\frac{1}{2} [\mathbf{J}^T \mathbf{J} + \lambda \mathbf{D}^T \mathbf{D}]^{-1} \mathbf{J}^T \boldsymbol{\Omega},\end{aligned}\tag{5}$$

where (Einstein summation implied)

$$\Omega_i = \frac{\partial^2 f(x_i, \beta)}{\partial \beta_j \partial \beta_k} \delta_1^j \delta_1^k \quad (6)$$

is the second directional derivative along δ_1 . From geometric considerations, δ_1 and δ_2 are also called the velocity and the accelerations terms. The acceleration term either magnifies or shrinks the velocity term, or tilts it in a different direction. We do not delve here into the theoretical foundations leading to Eq. (5) and its comparison to alternative ways of including second order corrections. The interested reader is referred to [2, 3, 4] for an in-depth discussion. We only make a few additional remarks for practical calculations.

Geodesic acceleration is most useful in a narrow canyon. When the algorithm has not yet found a canyon and is scanning a large plateau, which is where the initial guess is likely to be, then damping is small and δ_1 is large. This causes δ_2 to be even larger and pointing in the opposite direction, which may steer the algorithm in the wrong direction. To avoid this, we require the effect of the second order term to always be smaller than the first order term,

$$\frac{\sqrt{\delta_2 D \delta_2}}{\sqrt{\delta_1 D \delta_1}} < \alpha, \quad 0 \leq \alpha < 1, \quad (7)$$

where D has been included to ensure scale invariance.

Whether including the acceleration term pays off depends on the model and is ultimately for the user to test. With automatic differentiation the second directional derivative has about the same cost as the Jacobian, which is 3–4 times function evaluation.

Uphill steps. Accepting only downhill steps is the safest strategy, especially when one is far from the minimum. However, if the algorithm has to follow a narrow canyon, only very short steps would be acceptable, making the search costly. It might then be beneficial to conditionally also allow uphill steps. This can be thought of as analogous to the trajectory of a bobsled racer. A useful criterion for allowing an uphill step is

$$\begin{aligned} \beta_i &= \cos(\delta_{1i}, \delta_{1i-1}), \\ (1 - \beta_i)^b \chi_i^2 &< \min(\chi_1^2, \dots, \chi_{i-1}^2), \end{aligned} \quad (8)$$

where β_i is the cosine of the angle between the proposed step of the current iteration and that of the previous iteration. Only the velocity and not the acceleration term is considered here. According to Eq. (8), uphill steps are allowed only for acute angles, and the probability of acceptance increases with the alignment of δ_{1i} and δ_{1i-1} . The RHS of the inequality test usually refers to the sum of squares of the previous iteration, but we use the min function here in case any of the previous steps were uphill. Reasonable values for the exponent are 1 and 2, with $b = 2$ allowing greater uphill steps than $b = 1$.

In addition to faster convergence, this so-called “bold” acceptance criterion also exhibits a smaller probability of getting stuck at the local minima, which reside on the floor of the canyon.

The drawback of this criterion is the reduced stability of the algorithm, since allowing uphill steps makes it more susceptible to parameter evaporation. It is better suited for problems where the canyon is easily found, but navigating the canyon is the main difficulty. This is in contrast to problems that start in a difficult region of the parameter space, but once heading in the right direction, the minimum is easily found.

Updating the damping parameter. The simple scheme of raising and lowering λ by a factor of 10 as the steps are accepted or rejected is often adequate, but can be improved upon. Increasing λ by a small factor $\lambda_{\uparrow}$ when the step is uphill and decreasing it by a larger factor $\lambda_{\downarrow}$ when the step is downhill is expected to lead to a faster convergence (see Sec. VIII.C of [3]). The optimal values for the raising and lowering factors depend on the problem. One might start with, e.g., $\lambda_{\uparrow} = 3$ and $\lambda_{\downarrow} = 5$.

A more advanced method for updating the damping parameter makes use of the gain factor,

$$\rho = \frac{\chi^2(\mathbf{x}, \boldsymbol{\beta}) - \chi^2(\mathbf{x}, \boldsymbol{\beta} + \boldsymbol{\delta}_1)}{\chi^2(\mathbf{x}, \boldsymbol{\beta}) - |\mathbf{R} - 2(\mathbf{J} + \lambda \mathbf{D})\boldsymbol{\delta}_1|^2} = \frac{\chi^2(\mathbf{x}, \boldsymbol{\beta}) - \chi^2(\mathbf{x}, \boldsymbol{\beta} + \boldsymbol{\delta}_1)}{2\boldsymbol{\delta}_1(\mathbf{J}^T \mathbf{J} + \lambda \mathbf{D}^T \mathbf{D})\boldsymbol{\delta}_1}. \quad (9)$$

The second term in the denominator is the linearly predicted value of χ^2 using the new parameter vector $\boldsymbol{\beta} + \boldsymbol{\delta}_1$. A downhill step always produces a positive ρ and thus the steps can be accepted or rejected based on the sign of ρ . If ρ is close to zero, λ is updated with $\lambda_{\uparrow}$ and $\lambda_{\downarrow}$. However, if ρ is much greater than zero then the step should still be accepted but now the algorithm is stepping out of its trust-region. The next step is likely to increase χ^2 , which requires the lowering of λ . The oscillating behavior of λ can be avoided by limiting the decrease of λ so that the algorithm is unable to step out of the trust-region. Following Nielsen [5], this can be achieved by multiplying λ by $\max[1/\lambda_{\downarrow}, 1 - (2\rho - 1)^3]$ for $\rho > 0$. This method tends to increase the total number of iterations, but since the steps are accepted more often, χ^2 is calculated by a lesser number of times per iteration, which can overall speed up the procedure.

Another method for updating the damping parameter is due to Umrigar and Nightingale (unpublished). The idea is to vary λ based on the history of the success of previous steps. If the current step is accepted then λ is either decreased by a factor of ξ if the change in the direction of $\boldsymbol{\delta}_1$ is less than $\pi/2$, left unchanged if the change is more than $\pi/2$, or increased if the step was uphill:

$$\lambda_i = \lambda_{i-1}/\xi, \quad \cos(\boldsymbol{\delta}_{1i}, \boldsymbol{\delta}_{1i-1}) \geq 0, \quad (10)$$

$$\lambda_i = \lambda_{i-1}, \quad \cos(\boldsymbol{\delta}_{1i}, \boldsymbol{\delta}_{1i-1}) < 0, \quad (11)$$

$$\lambda_i = \lambda_{i-1}\sqrt{\xi}, \quad \chi_i^2 \geq \chi_{i-1}^2. \quad (12)$$

If the step is rejected, then λ is always increased, but the increase depends on the change of $\boldsymbol{\delta}_1$:

$$\lambda_i = \lambda_{i-1} \sqrt{\xi}, \quad \cos(\boldsymbol{\delta}_{1i}, \boldsymbol{\delta}_{1i-1}) \geq 0, \quad (13)$$

$$\lambda_i = \lambda_{i-1} \xi, \quad \cos(\boldsymbol{\delta}_{1i}, \boldsymbol{\delta}_{1i-1}) < 0. \quad (14)$$

ξ is constrained to the range $1 \dots 100$ and is given by

$$\xi = (1 - |2a - 1|)^{-2}, \quad (15)$$

where a is updated according to

$$a_i = a_{i-1}m + A(1 - m), \quad (16)$$

where

$$A = 1 \quad \cos(\boldsymbol{\delta}_{1i}, \boldsymbol{\delta}_{1i-1}) \geq 0 \quad \text{and} \quad \chi_i^2 < \chi_{i-1}^2, \quad (17)$$

$$A = 1/2, \quad \cos(\boldsymbol{\delta}_{1i}, \boldsymbol{\delta}_{1i-1}) < 0 \quad \text{or} \quad \chi_i^2 \geq \chi_{i-1}^2, \quad (18)$$

$$A = 0 \quad \text{if step was rejected.} \quad (19)$$

The method works well in some cases but has thus far not seen extensive testing. In GADfit, the initial values for a and m are set to 0.5 and $e^{-0.2}$.

Convergence criteria. At the solution, the residual vector $\mathbf{y} - \mathbf{f}$ is orthogonal to the range of the Jacobian, $\mathbf{J}\boldsymbol{\delta}_1$. A measure of convergence can then be taken as the cosine of the angle between these two quantities,

$$|\cos \phi| = \frac{|\mathbf{R}\mathbf{J}\boldsymbol{\delta}_1|}{|\mathbf{R}||\mathbf{J}\boldsymbol{\delta}_1|}. \quad (20)$$

In addition to (20), most of the standard convergence criteria are available in GADfit. See the sections for input parameters to the fitting procedure for more details.

Robust cost functions In order to damp the effect of outliers in the data, instead of minimizing the sum of squared differences [Eq. (1)], we can minimize the quantity

$$\sum_i \rho \left[\left(\frac{y_i - f(x_i, \boldsymbol{\beta})}{\sigma_i} \right)^2 \right] \quad (21)$$

instead. This leads to the following modifications of the optimization algorithm:

$$J_{ij} = \sqrt{\frac{\partial \rho(z)}{\partial z_i}} \frac{1}{\sigma_i} \frac{\partial f(x_i, \boldsymbol{\beta})}{\partial \beta_j}, \quad (22)$$

$$R_i = \sqrt{\frac{\partial \rho(z)}{\partial z_i}} \frac{y_i - f(x_i, \boldsymbol{\beta})}{\sigma_i},$$

where $\partial\rho(z)/\partial z_i$ is shorthand for

$$\left. \frac{\partial\rho(z)}{\partial z} \right|_{z=[(y_i-f(x_i,\boldsymbol{\beta}))/\sigma_i]^2}.$$

The linear case, $\rho(z) = z$, reduces to the standard least squares problem. Another option is to use the Cauchy loss function, $\rho(z) = \ln(1 + z)$, which severely weakens the influence of outliers.

2.2 Global nonlinear optimization

When fitting many curves simultaneously with shared parameters, the data sets are stacked to form a single data set, which is then fitted with a piecewise function. The fitting function is constructed such that some parameters only have an effect on a subset of data points (local parameters) whereas others are active over the whole data range (global parameters).

As an example, take two data sets with sizes n and m and a fitting function $f(x, \alpha, \beta)$, which depends on two parameters α and β . α is different for the two curves whereas β is shared. For instance, for an exponential decay process these could be different initial amplitudes but the same decay constant. The global Jacobian is then

$$\mathbf{J} = \begin{pmatrix} \frac{1}{\sigma_1} \frac{\partial f(x_1, \alpha, \beta)}{\partial \beta} & \frac{1}{\sigma_1} \frac{\partial f(x_1, \alpha, \beta)}{\partial \alpha_1} & 0 \\ \dots & \dots & \dots \\ \frac{1}{\sigma_n} \frac{\partial f(x_n, \alpha, \beta)}{\partial \beta} & \frac{1}{\sigma_n} \frac{\partial f(x_n, \alpha, \beta)}{\partial \alpha_1} & 0 \\ \frac{1}{\sigma_{n+1}} \frac{\partial f(x_{n+1}, \alpha, \beta)}{\partial \beta} & 0 & \frac{1}{\sigma_{n+1}} \frac{\partial f(x_{n+1}, \alpha, \beta)}{\partial \alpha_2} \\ \dots & \dots & \dots \\ \frac{1}{\sigma_{n+m}} \frac{\partial f(x_{n+m}, \alpha, \beta)}{\partial \beta} & 0 & \frac{1}{\sigma_{n+m}} \frac{\partial f(x_{n+m}, \alpha, \beta)}{\partial \alpha_2} \end{pmatrix}, \quad (23)$$

where the notation is $\partial f / \partial \alpha_i \equiv \partial f / \partial \alpha|_{\alpha=\alpha_i}$. For easier implementation, the global parameters always occupy the first columns in the Jacobian. Similarly, with 4 data sets, each consisting of 100 points, and with 4 local and 2 global parameters, the Jacobian would have the dimensions 400×18 . Sparsity of the Jacobian is not exploited. Since no conceptual change is introduced, the fitting procedure follows the same algorithm as with a single curve.

2.3 Automatic differentiation

If the Jacobian (23) was calculated using finite differences, the maximum accuracy one can hope for is half the number of significant digits corresponding to machine precision (about 7 for double precision). This is often sufficient if the fitting function is a simple combination of elementary functions such as the Gaussian or an exponential. However, problems can arise with difficult functions such as those containing a highly nonlinear combination of elementary or special functions or those requiring numerical integration. In those situations, both accuracy and the computational cost of finite differences can become problematic.

Another thing to keep in mind is that with finite differences, the evaluation of J_{ij} always takes two function calls and the evaluation of the whole gradient $(J_{i1}, \dots, J_{in})$ takes $2n$ function calls.

In contrast, with automatic differentiation the derivatives are obtained with the same accuracy as function evaluation itself, while the computational cost is 3–4 times that of function evaluation *irrespective* of the number of parameters [6]. AD exploits the fact that any mathematical function, no matter how complicated, is executed as a sequence of elementary arithmetic operations on a computer. By applying the chain rule of calculus at each step of execution, the derivative of a function can be computed automatically, with working accuracy, and using at most a small constant of the computer time required for function evaluation. It should be noted that while AD is not numerical differentiation, it is also not symbolic differentiation. The chain rule is applied directly to the results of elementary operations and the symbolic form of the full derivative is never stored in any way. Only the derivatives of the elementary functions need to be coded by hand (in AD literature, the term “elemental function” is more common). Afterwards, the algorithm can be applied to functions of arbitrary complexity.

In this document, we describe two flavors of AD. The forward mode of AD is well-suited for calculating the gradient of a multi-valued function depending on a single parameter (just the derivative in this case) or for calculating the directional derivative of a single-valued function depending on many parameters. The reverse mode is suitable for calculating the gradient of a single-valued function depending on many parameters. For the general case of n input and m output variables, a nontrivial mix of both modes would in principle be optimal.

2.3.1 Forward mode

The basic idea behind the forward mode is to extend all numbers to include a second component,

$$x \rightarrow \tilde{x} = x + \dot{x}d \equiv (x, \dot{x}), \quad (24)$$

resulting in so-called dual numbers. The arithmetic on dual numbers is defined by requiring $d^2 = 0$. This is in contrast to ordinary complex numbers, which are also dual numbers but where the second component is defined according to $i^2 = -1$. Elementary arithmetic between dual numbers can then be written as follows:

$$\begin{aligned} \tilde{x} + \tilde{y} &= x + y + (\dot{x} + \dot{y})d = (x + y, \dot{x} + \dot{y}), \\ \tilde{x}\tilde{y} &= xy + x\dot{y}d + \dot{x}yd + \dot{x}\dot{y}d^2 = (xy, \dot{x}y + x\dot{y}), \\ \frac{\tilde{x}}{\tilde{y}} &= \frac{x + \dot{x}d}{y + \dot{y}d} = \frac{(x + \dot{x}d)(y - \dot{y}d)}{y^2 - \dot{y}^2d^2} = \left(\frac{x}{y}, \frac{\dot{x}y - x\dot{y}}{y^2} \right), \\ \sin \tilde{x} &= \dots = (\sin x, \dot{x} \cos x), \end{aligned} \quad (25)$$

where the last equality follows after a series expansion. Any ordinary number can be interpreted as $\tilde{x} = x + 0d$. It is seen from Eqs. (25) that the second

component, also called the tangent, always has the form of the derivative of the operation. In fact, it can be proven that for any function $f(x)$, with a dual number as its argument,

$$f(\tilde{x}) = f(x) + f'(x)\dot{x}d = (f(x), f'(x)\dot{x}), \quad (26)$$

where $f'(x)$ is the derivative of f evaluated at x . Applying the dual number arithmetic to the composition of f and g , we get

$$f(g(\tilde{x})) = f(g(x) + g'(x)\dot{x}d) = (f(g(x)), f'(g(x))g'(x)\dot{x}). \quad (27)$$

With Eq. (27) we have actually rediscovered the chain rule! Repeatedly applying Eq. (27) yields the derivatives of arbitrarily complex functions.

Which derivative the dot symbol represents depends on the initial values of the tangents. It can be shown that in general the initial values define the direction of the derivative in the space of independent variables. For instance, if we have two independent variables x and y and require the derivative with respect to x , then the tangent vector is $(1, 0)$ in the xy -plane and the variables should be seeded with $\dot{x} = dx/dx = 1$ and $\dot{y} = dy/dx = 0$. For a directional derivative $\nabla_{x,y}f(x,y)\mathbf{v}$, where $\mathbf{v} = (\alpha, \beta)$, the seeds would be $\dot{x} = \alpha$ and $\dot{y} = \beta$.

As a simple example, we shall evaluate $\frac{\partial}{\partial x}f(x,y)$ for $f(x,y) = xy + \sin(xy)$ at (x_0, y_0) . Applying the rules of dual numbers, the computational graph for this procedure, in terms of the intermediate results $\tilde{v}_i$, is

$$\begin{aligned} \tilde{v}_1 &= (x_0, \dot{x}) = (x_0, 1), \\ \tilde{v}_2 &= (y_0, \dot{y}) = (y_0, 0), \\ \tilde{v}_3 &= \tilde{v}_1 \tilde{v}_2 = (v_1 v_2, v_1 \dot{v}_2 + \dot{v}_1 v_2) = (x_0 y_0, y_0), \\ \tilde{v}_4 &= \sin \tilde{v}_3 = (\sin v_3, \dot{v}_3 \cos v_3) = (\sin x_0 y_0, y_0 \cos(x_0 y_0)), \\ \tilde{v}_5 &= \tilde{v}_3 + \tilde{v}_4 = (x_0 y_0 + \sin x_0 y_0, y_0 + y_0 \cos(x_0 y_0)). \end{aligned} \quad (28)$$

For such a simple case, it is easy to analytically check that the tangent of $\tilde{v}_5$ is indeed the correct result. Note that AD is as economical as the function evaluation in terms of reusing intermediate results — the derivative of xy (the tangent of $\tilde{v}_3$) is computed only once even though it appears in two places. We remind that AD is not symbolic differentiation since there is no record of the symbolic form of the derivatives. The algorithm only needs to recognize the elemental operation at hand, whose derivative needs to be precoded, but otherwise the procedure is completely mechanical. At each step in the computational graph, the algorithm has forgotten how it got there. For a better illustration, here is a throwaway C++ implementation of the same calculation with $(x_0, y_0) = (1.5, 0.7)$.

```
class ddouble {
public:
    double val, dot;
    ddouble(double val, double dot) : val { val }, dot { dot } {}
};
```

```

auto operator+(ddouble x, ddouble y) -> ddouble {
    return ddouble(x.val + y.val, x.dot + y.dot);
}
auto operator*(ddouble x, ddouble y) -> ddouble {
    return ddouble(x.val * y.val, x.val * y.dot + x.dot * y.val);
}
auto sin(ddouble x) -> ddouble {
    return ddouble(sin(x.val), x.dot * cos(x.val));
}
int main() {
    ddouble x(1.5, 1);
    ddouble y(0.7, 0);
    ddouble f;
    f = x * y + sin(x * y);
    std::cout << f.dot; // 1.0483
}

```

If, instead of just $\frac{\partial}{\partial x}$, the whole gradient $\nabla_{x,y}f(x,y)$ is required, a second calculation needs to be performed starting with $\dot{x} = 0, \dot{y} = 1$. The accuracy is the same as before, but the cost is twice as much. Note that for each partial derivative, the values (first components) of the elemental operations are the same and could be reused. Due to the $O(n)$ scaling, the forward mode is generally not used for gradient calculation. Instead, it is better suited for directional derivatives, where a single sweep produces the value of $\nabla_{\mathbf{p}}f(\mathbf{p})\mathbf{v}$, where $\mathbf{p}$ is a vector of independent variables.

In GADfit, the forward mode comes into play in only one place, which is the acceleration term (5) which contains a second order directional derivative (6). The generalization of the forward mode to second order is straightforward. The AD numbers now contain three components,

$$\tilde{x} = x + \dot{x}d + \ddot{x}d^2, \quad (29)$$

and the arithmetic is defined by requiring third and higher order terms to be zero, $d^3 = 0$. The elemental operations become somewhat more complex,

$$\begin{aligned}
\tilde{x} + \tilde{y} &= x + y + (\dot{x} + \dot{y})d + (\ddot{x} + \ddot{y})d^2, \\
\tilde{x}\tilde{y} &= xy + (\dot{x}y + x\dot{y})d + (\ddot{x}y + 2\dot{x}\dot{y} + x\ddot{y})d^2, \\
\frac{\tilde{x}}{\tilde{y}} &= \frac{x}{y} + \frac{\dot{x}y - x\dot{y}}{y^2}d + \frac{\ddot{x}y - x\ddot{y} - 2\dot{x}\dot{y} + 2\frac{x\dot{y}^2}{y}}{y^2}d^2, \\
\sin \tilde{x} &= \sin x + \dot{x} \cos x d + (\ddot{x} \cos x - \dot{x}^2 \sin x) d^2,
\end{aligned} \quad (30)$$

but otherwise the algorithm has the same structure. The complexity is actually less than appears in Eqs. (30), since once the first component of the resulting number has been calculated, it can be immediately used in the calculation of the second component. And for the third component, both of the previous terms

are available. For instance, the last two of Eqs. (30) can be rewritten in the form

$$\begin{aligned}\tilde{v} &= \frac{\tilde{x}}{\tilde{y}} = v + \frac{\dot{x} - v\dot{y}}{y}d + \frac{\ddot{x} - v\ddot{y} - 2\dot{v}\dot{y}}{y}d^2, \\ \tilde{v} &= \sin \tilde{x} = v + \dot{x} \cos xd + (\ddot{x}\dot{v}/\dot{x} - \dot{x}^2v)d^2.\end{aligned}\tag{31}$$

In order to compute the second directional derivative $\partial_i\partial_j f(x; \boldsymbol{\beta})v^i v^j$, where $\boldsymbol{\beta}$ is the parameter vector and $\partial_i \equiv \partial/\partial\beta_i$, the parameters should be seeded with $(\beta_i, v_i, 0)$. A single sweep then produces the function value and the directional first and second derivatives.

2.3.2 Reverse mode

The reverse mode is generally considered superior to the forward mode because it is more efficient for calculating the gradient which is more often required than a directional derivative. The Jacobian is exactly such a quantity, where each row $(J_{i1}, \dots, J_{in})$ is the gradient of a single-valued function $f(x_i, \boldsymbol{\beta})$. At the same time, it is also harder to implement since it consists of two parts — a forward sweep, which performs function evaluation, and a return sweep, which processes the computational graph in reverse order to yield the partial derivatives. Information about the elemental operations must be saved during the forward sweep which necessitates some form of memory management. This is in contrast to the forward mode where both the function value and the derivatives are obtained by traversing in the same direction along the computational graph and no additional memory needs to be used.

The reverse mode is best understood with an example. The sequence of elemental operations for the evaluation of $f(x, y) = xy + \sin \frac{x}{y}$ at the point $(2.3, 0.8)$, which we call the forward sweep, is

$$v_1 = x = 2.3000, \tag{32a}$$

$$v_2 = y = 0.8000, \tag{32b}$$

$$v_3 = v_1 v_2 = 1.8400, \tag{32c}$$

$$v_4 = v_1/v_2 = 2.8750, \tag{32d}$$

$$v_5 = \sin v_4 = 0.2634, \tag{32e}$$

$$v_6 = v_3 + v_5 = 2.1034, \tag{32f}$$

where the first two operations are just variable initialization. While the order of some of these operations can be compiler specific, it does not influence the outcome of the AD method. One might suppose that in order to compute the gradient $\nabla_{x,y} f$, it is necessary to process each of Eqs. (32) twice, once for either parameter. However, the number of operations can be lowered by applying the chain rule backwards, i.e., by processing Eqs. (32) in the order $v_6 \dots v_1$ in terms of the adjoint quantities $\bar{v}_i \equiv \partial f/\partial v_i$. This is called the return sweep. Two trivial results are then immediately obtained from Eq. (32f):

$$\bar{v}_3 = \frac{\partial f}{\partial v_3} = \frac{\partial v_6}{\partial v_3} = 1, \quad \bar{v}_5 = \frac{\partial f}{\partial v_5} = \frac{\partial v_6}{\partial v_5} = 1. \tag{33a}$$

Using the result for $\bar{v}_5$, Eq. (32e) then yields the adjoint of v_4 ,

$$\bar{v}_4 = \frac{\partial v_6}{\partial v_4} = \frac{\partial v_6}{\partial v_5} \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \cos v_4 = 1 \times (-0.9647) = -0.9647. \quad (33b)$$

Next, we process the two components of Eq. (32d),

$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \frac{\bar{v}_4}{v_2} = -1.2059, \quad \bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = -\frac{\bar{v}_4 v_1}{v_2^2} = 3.4669. \quad (33c)$$

From Eq. (32c),

$$\bar{v}_1 = \bar{v}_1 + \bar{v}_3 \frac{\partial v_3}{\partial v_1} = -0.4059, \quad \bar{v}_2 = \bar{v}_2 + \bar{v}_3 \frac{\partial v_3}{\partial v_2} = 5.7669. \quad (33d)$$

The computation of $\bar{v}_i$ is cumulative. If some variable v_i appears several times in the forward sweep then all occurrences of v_i must be accounted for. This is seen in Eqs. (33d), which remember the previous values of $\bar{v}_1$ and $\bar{v}_2$. With the above scheme, Eqs. (32a) and (32b) would trivially yield $\bar{v}_1 = \bar{v}_1$ and $\bar{v}_2 = \bar{v}_2$ and are not processed in practice. We thus have the result

$$\nabla_{x,y} f(x, y) = (-0.4059, 5.7669). \quad (34)$$

For such a simple example finite differences would of course produce the same result. However, some benefit of the reverse mode is already seen — a single sweep produces the partial derivatives of *both* variables. This property becomes especially advantageous for optimization problems with a large number of fitting parameters.

The algorithm can now be formulated as shown in Table 1. The first n oper-

Table 1: The reverse mode of AD. $a += b$ is a shorthand for $a = a + b$.

Forward sweep	Return sweep
$v_i = x_i, \quad i = 1 \dots n$	$\partial f \partial x_i = \bar{v}_i, \quad i = 1 \dots n$
$v_i += g_i(v_{j \rightsquigarrow i}), \quad i = n+1 \dots n+m$	$\bar{v}_i += \bar{v}_{j \rightsquigarrow i} \partial v_{j \rightsquigarrow i} / \partial v_i$
$f = v_{n+m}$	$\bar{v}_{n+m} = \partial f / \partial v_{n+m} = 1$

ations of the forward sweep initialize n independent variables. The intermediate values v_i are then calculated by applying the elemental operations g_i to all v_j of which v_i directly depends on (denoted with $\rightsquigarrow$). The last one of the m intermediate values is the function value, $f = v_{n+m}$. Information about each elemental operation during the forward sweep must be saved as it is required during the return sweep. The return sweep moves in the opposite direction, starting with $\partial f / \partial v_{n+m} = \partial f / \partial f = 1$ and followed by the calculation of $\bar{v}_i$ as illustrated in the above example. The partial derivatives are given by the first n values of the adjoints array $\bar{\mathbf{v}}$.

2.3.3 Implementation

GADfit comes in two implementations — in C++ and in Fortran, both based on operator overloading. The basic variable is the “AD variable”,

```
// C++
class AdVar
{
public:
    double val, d, dd;
    int idx;
    ...
};

! Fortran
type advar
    real(kp) :: val, d = 0.0_kp, dd = 0.0_kp
    integer :: index = 0
contains
    ...
end type advar
```

This variable type is used for both forward and reverse modes. Each instance has fields for the value and first and second derivatives, and an index field. The former three are double or quadruple (Fortran only) precision floating point numbers. Index is zero for “passive” variables, for which differentiation is not required, and is nonzero otherwise. Which mode is currently in use is determined by the sign of the index variable (`idx` or `index`). Whereas the `d` and `dd` fields are part of the forward mode only, the index field is required for both modes. For the forward mode, the reason to use the index is to immediately identify whether the variable is passive or active and thus many calculations involving `d=0.0` and `dd=0.0` can be avoided. In the forward mode, it only matters whether the index is zero or nonzero, whereas in the reverse mode, each new active variable is given a unique index.

The reverse mode is complicated by the fact that during the forward sweep each elemental operation² must be recorded by saving the indices of the participating variables and of the operation code into an array called the execution trace. The intermediate (forward) values are saved into a separate array. As an example, if the variables a and b had indices 4 and 5, and the operation code for addition were 23, then for the operation $c = a + b$ that section of the trace would read $(\dots, 4, 5, 6, 23, \dots)$, where 6 is the index of the new variable c . For unary operations, there would be 3 entries in the trace, $(\dots, \text{in}, \text{out}, \text{opcode}, \dots)$, for ternary operations 5 entries and so on. At the beginning of each return sweep, the adjoints array is reinitialized to zero with the last element set equal to 1. The execution trace is then processed starting with the last element, and the adjoints

²Not to be confused with the Fortran “elemental” keyword. In fact, these are all impure non-elemental functions.

are calculated according to the AD algorithm, first by identifying the relevant operation (`switch(op_id)` or `case select(op_id)`). To summarize, the main AD work variables in the reverse mode, along with their internal names, are

- the intermediate values (`reverse_work::forwards` or `forward_values`),
- the adjoints (`reverse_work::adjoints` or `adjoints`),
- the execution trace (`reverse_work::trace` or `trace`),
- the constants (`reverse_work::constants` or `ad_constants`).

No work arrays are required for the forward mode. The current list of elemental operations includes

- $-x_1, \quad x_1 \pm x_2, \quad x_1 x_2, \quad \frac{x_1}{x_2}, \quad \exp(x_1), \quad \sqrt{x_1}, \quad x_1^{x_2}, \quad |x_1|, \quad \ln x_1,$
- $\sin x_1, \quad \cos x_1, \quad \tan x_1, \quad \sinh x_1, \quad \tanh x_1, \quad \operatorname{asin} x_1, \quad \operatorname{acos} x_1,$
 $\operatorname{atan} x_1, \quad \operatorname{asinh} x_1, \quad \operatorname{acosh} x_1, \quad \operatorname{atanh} x_1,$
- $\operatorname{erf}(x_1) = \frac{2}{\sqrt{\pi}} \int_0^{x_1} e^{-t^2} dt, \quad \operatorname{Li}_2(x_1) = - \int_0^{x_1} \frac{\ln(1-u)}{u} du,$
- $\int_{a(x_1, \dots, x_n)}^{b(x_1, \dots, x_n)} f(x_1, \dots, x_n; t) dt,$

where x_i is either an AD variable, a real number of single/double/quadruple precision, or an integer. Integrals can be nested up to two layers, i.e., only single and double integrals are allowed. New elemental operations can be added to the source code without too much effort. Alternatively, any requests/suggestions are most welcome at <https://github.com/raullaasner/gadfit/issues>. Unless the proposed elemental operation depends on a nonstandard external library, we will make an effort to include it in the next release.

2.3.4 AD with numerical integration

GADfit supports fitting functions containing bounded, semi-infinite, and infinite integrals. Numerical integration is performed using the adaptive Gauss-Kronrod algorithm. Default is the 15-point rule; also available are 21, 31, 41, 51, and 61 point rules. The integration interval is divided into subintervals, and on each iteration the subinterval with the largest error is processed. This is similar to what QUADPACK does except without the epsilon algorithm. During this procedure, AD is temporarily switched off by making all variables passive. When the integration procedure has converged with the desired accuracy, AD is switched on and the integral is computed once more with an optimal set of subintervals. Note that the optimal set of subintervals satisfies the error tolerances of the original integral, not necessarily the integral over the derivative of the integrand. One might suppose then that the procedure needs to be performed separately for each parameter, presumably using the forward mode of AD, yielding a unique

set of subintervals for each parameter. Strictly speaking though, the derivative is required for the approximation

$$F(\mathbf{p}) = \int_A^B f(\mathbf{p}, x) dx \approx \sum_j \frac{b_j - a_j}{2} \sum_i w_{i,j} f\left(\mathbf{p}, \frac{b_j - a_j}{2} \xi_{i,j} + \frac{a_j + b_j}{2}\right), \quad (35)$$

where a_j and b_j are the bounds of each subinterval, $w_{i,j}$ are the Gauss-Kronrod weights, and $\xi_{i,j}$ are the roots of the Legendre polynomials. Here, for brevity, $\mathbf{p}$ includes both the fitting parameters and optionally the independent variables [x_i in Eq. (1)]. Since it is Eq. (35) that is the fitting function (or part of it) and not the exact integral, differentiation should be performed using the same approximate form instead of finding a more accurate approximation for the integral over the derivative of f by using a different set of subintervals.

The dependence on one or more fitting parameters can be in the integrand and/or one or both bounds. The general formula for the gradient is

$$\begin{aligned} \nabla_{\mathbf{p}} \int_{a(\mathbf{p})}^{b(\mathbf{p})} f(\mathbf{p}, x) dx \\ = f(\mathbf{p}, b(\mathbf{p})) \nabla_{\mathbf{p}} b(\mathbf{p}) - f(\mathbf{p}, a(\mathbf{p})) \nabla_{\mathbf{p}} a(\mathbf{p}) + \int_{a(\mathbf{p})}^{b(\mathbf{p})} \nabla_{\mathbf{p}} f(\mathbf{p}, x) dx, \end{aligned} \quad (36)$$

where $\mathbf{p}$ is the fitting parameter vector.

The general formula for the second derivative, required for the acceleration term, is

$$\begin{aligned} \nabla_{\mathbf{p}}^2 \int_{a(\mathbf{p})}^{b(\mathbf{p})} f(\mathbf{p}, x) dx &= f(\mathbf{p}, b(\mathbf{p})) \nabla_{\mathbf{p}}^2 b(\mathbf{p}) - f(\mathbf{p}, a(\mathbf{p})) \nabla_{\mathbf{p}}^2 a(\mathbf{p}) \\ &\quad + \left[\nabla_{\mathbf{p}} f(\mathbf{p}, b(\mathbf{p})) + \nabla_{\chi} f(\chi, b(\mathbf{p}))|_{\chi=\mathbf{p}} \right] \nabla_{\mathbf{p}} b(\mathbf{p}) \\ &\quad - \left[\nabla_{\mathbf{p}} f(\mathbf{p}, a(\mathbf{p})) + \nabla_{\chi} f(\chi, a(\mathbf{p}))|_{\chi=\mathbf{p}} \right] \nabla_{\mathbf{p}} a(\mathbf{p}) + \int_{a(\mathbf{p})}^{b(\mathbf{p})} \nabla_{\mathbf{p}}^2 f(\mathbf{p}, x) dx, \end{aligned} \quad (37)$$

The above scheme could be generalized to any order, but it is probably not worth going deeper than double integrals since each integration level greatly reduces accuracy. Even nested double integrals should be avoided if possible.

We also support direct double integrals with the 2D quadrature given as

$$\begin{aligned} F(\mathbf{p}) &= \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y) dx dy \\ &\approx \sum_j \frac{x_{2j} - x_{1j}}{2} \frac{y_{2j} - y_{1j}}{2} \sum_i w_{i,j}^{2D} \\ &\quad \times f\left(\mathbf{p}, \frac{x_{2j} - x_{1j}}{2} \xi_{i,j} + \frac{x_{2j} + x_{1j}}{2}, \frac{y_{2j} - y_{1j}}{2} \xi_{i,j} + \frac{y_{2j} + y_{1j}}{2}\right), \end{aligned} \quad (38)$$

where $w_{i,j}^{2D}$ are products of the 1D weights. The general formula for the gradient is

$$\begin{aligned}
& \nabla_{\mathbf{p}} \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y) \, dx \, dy \\
&= [\nabla_{\mathbf{p}} y_2(\mathbf{p})] \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y_2(\mathbf{p})) \, dx - [\nabla_{\mathbf{p}} y_1(\mathbf{p})] \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y_1(\mathbf{p})) \, dx \\
&\quad + [\nabla_{\mathbf{p}} x_2(\mathbf{p})] \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} f(\mathbf{p}, x_2(\mathbf{p}), y) \, dy - [\nabla_{\mathbf{p}} x_1(\mathbf{p})] \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} f(\mathbf{p}, x_1(\mathbf{p}), y) \, dy \\
&\quad + \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} \nabla_{\mathbf{p}} f(\mathbf{p}, x, y) \, dx \, dy.
\end{aligned} \tag{39}$$

The general formula for the second derivative is

$$\begin{aligned}
& \nabla_{\mathbf{p}}^2 \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y) \, dx \, dy = \xi_{Y2} - \xi_{Y1} + \xi_{X2} - \xi_{X1} \\
&\quad + \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} \nabla_{\mathbf{p}}^2 f(\mathbf{p}, x, y) \, dx \, dy,
\end{aligned} \tag{40}$$

where

$$\begin{aligned}
\xi_{Y1} &= [\nabla_{\mathbf{p}}^2 y_1(\mathbf{p})] \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y_1(\mathbf{p})) \, dx + [\nabla_{\mathbf{p}} y_1(\mathbf{p})] \\
&\quad \times \left[[\nabla_{\mathbf{p}} x_2(\mathbf{p})] f(\mathbf{p}, x_2(\mathbf{p}), y_1(\mathbf{p})) - [\nabla_{\mathbf{p}} x_1(\mathbf{p})] f(\mathbf{p}, x_1(\mathbf{p}), y_1(\mathbf{p})) \right. \\
&\quad \left. + \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} \nabla_{\chi} f(\chi, x, y_1(\mathbf{p}))|_{\chi=\mathbf{p}} \, dx + \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} \nabla_{\mathbf{p}} f(\mathbf{p}, x, y_1(\mathbf{p})) \, dx \right], \\
\xi_{Y2} &= [\nabla_{\mathbf{p}}^2 y_2(\mathbf{p})] \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} f(\mathbf{p}, x, y_2(\mathbf{p})) \, dx + [\nabla_{\mathbf{p}} y_2(\mathbf{p})] \\
&\quad \times \left[[\nabla_{\mathbf{p}} x_2(\mathbf{p})] f(\mathbf{p}, x_2(\mathbf{p}), y_2(\mathbf{p})) - [\nabla_{\mathbf{p}} x_1(\mathbf{p})] f(\mathbf{p}, x_1(\mathbf{p}), y_2(\mathbf{p})) \right. \\
&\quad \left. + \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} \nabla_{\chi} f(\chi, x, y_2(\mathbf{p}))|_{\chi=\mathbf{p}} \, dx + \int_{x_1(\mathbf{p})}^{x_2(\mathbf{p})} \nabla_{\mathbf{p}} f(\mathbf{p}, x, y_2(\mathbf{p})) \, dx \right], \\
\xi_{X1} &= [\nabla_{\mathbf{p}}^2 x_1(\mathbf{p})] \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} f(\mathbf{p}, x_1(\mathbf{p}), y) \, dy + [\nabla_{\mathbf{p}} x_1(\mathbf{p})] \\
&\quad \times \left[[\nabla_{\mathbf{p}} y_2(\mathbf{p})] f(\mathbf{p}, x_1(\mathbf{p}), y_2(\mathbf{p})) - [\nabla_{\mathbf{p}} y_1(\mathbf{p})] f(\mathbf{p}, x_1(\mathbf{p}), y_1(\mathbf{p})) \right. \\
&\quad \left. + \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \nabla_{\chi} f(\chi, x_1(\mathbf{p}), y)|_{\chi=\mathbf{p}} \, dy + \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \nabla_{\mathbf{p}} f(\mathbf{p}, x_1(\mathbf{p}), y) \, dy \right], \\
\xi_{X2} &= [\nabla_{\mathbf{p}}^2 x_2(\mathbf{p})] \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} f(\mathbf{p}, x_2(\mathbf{p}), y) \, dy + [\nabla_{\mathbf{p}} x_2(\mathbf{p})] \\
&\quad \times \left[[\nabla_{\mathbf{p}} y_2(\mathbf{p})] f(\mathbf{p}, x_2(\mathbf{p}), y_2(\mathbf{p})) - [\nabla_{\mathbf{p}} y_1(\mathbf{p})] f(\mathbf{p}, x_2(\mathbf{p}), y_1(\mathbf{p})) \right. \\
&\quad \left. + \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \nabla_{\chi} f(\chi, x_2(\mathbf{p}), y)|_{\chi=\mathbf{p}} \, dy + \int_{y_1(\mathbf{p})}^{y_2(\mathbf{p})} \nabla_{\mathbf{p}} f(\mathbf{p}, x_2(\mathbf{p}), y) \, dy \right].
\end{aligned} \tag{41}$$

3 Building and linking

GADfit is a library, i.e., not a standalone program, written in *two* different compiled languages. It started out as a Fortran implementation and was later duplicated in C++. The Fortran development has largely stopped and only the C++ implementation sees progress these days. What this means is that the user is expected to have at least basic knowledge of C++ or Fortran. Specifically, the fitting functions and the driver code would need to be written in C++/Fortran. The structure of the code is such that it is currently not possible to easily interface it with other languages. The C++/Fortran requirement might seem inconvenient for the user, but it allows to make full use of a powerful programming language for writing functions of arbitrary complexity, which is the primary benefit of using AD.

3.1 Building process

GADfit is dependent on the CMake build system generator. CMake is a set of tools for configuring, building, and testing software. It is released under the New BSD License and can be installed by issuing

```
sudo apt-get install cmake
```

(Here and elsewhere, we demonstrate how to obtain the relevant packages on Ubuntu; advanced users can skip this section.) For building from source visit <http://cmake.org>. GADfit is developed with the requirement that it should work with the GNU compilers. In principle, any C++ 20 or Fortran 2008 conforming compiler should also work.

The overall build process is similar for the C++ and Fortran versions, although they do have different prerequisites (see Secs. 3.1.1 and 3.1.2 for details). Here we lay out instructions that apply for both.

Steps for configuring and building GADfit are the following. Untar the source code and navigate into the build directory (`~build`)

```
tar xf gadfit.tar.gz
mkdir build && cd build
```

One can also build in the source directory (`~gadfit`) but it is generally a good habit to do out-of-source builds or at least create a separate build directory within the source directory.

For configuring, you can use either a graphical CMake front end, a text file containing the configuration variables, or specify the build environment from the command line (`cmake <options> ...`). The two commonly used graphical front ends are the command line based `ccmake`, obtained by installing `cmake-curses-gui`, and the Qt-based `cmake-gui`, obtained by installing `cmake-qt-gui`. When using `ccmake`, issue

```
cmake ~gadfit
ccmake .
```

from the build directory. Some CMake variables and options appear, most of which should be self-explanatory. A short help text to each variable is displayed at the bottom in a status bar. Pressing 't' reveals all options. When done editing, press 'c' to reconfigure and 'g' to generate the native build script (the unix makefile on Linux). Pay attention when `ccmake` warns you that the cache variables have been reset. This will happen, e.g., when changing the compiler, and will necessitate the reconfiguring of some variables. After configuring, it is a good idea to go through all the variables once more to check that everything is correct (see Sec. 3.1.3 for configuration variables). When finished, exit `ccmake` and issue

```
cmake --build .
cmake --build . --target test
cmake --build . --target install
```

The default build system generated by CMake is GNU makefiles on Linux. A different build system can be chosen by passing an argument to the CMake generator function. For instance, for using Ninja, use `-G Ninja` during the initial configuring,

```
cmake -G Ninja ~gadfit
```

In order to use `cmake-gui` instead of `ccmake`, start with

```
cmake ~gadfit
cmake-gui .
```

The rest is analogous.

The CMake configuration variables can also be put into an external file, an example of which is `initial_cache.cmake.example` in the root directory of GADfit. It can be used as follows:

```
cp initial_cache.cmake.example initial_cache.cmake
# Edit initial_cache.cmake to reflect your environment
cmake -C ~gadfit/initial_cache.cmake ~gadfit
```

from the build directory, followed by the build commands. A change of the content of the configuration file has no effect after the first configuring. Instead, it would be necessary to empty `~build` before using the configuration file again or, better, use `ccmake`.

3.1.1 C++ prerequisites

- A C++ compiler. The GNU C++ compiler (g++) is usually pre-installed on a Linux distribution. If not, visit gcc.gnu.org to build and install one manually.
- spdlog, which is a C++ logger. Install it either manually, using your distribution's package manager, or do nothing in which case the build system downloads and builds it automatically.

3.1.2 Fortran prerequisites

- A Fortran compiler GFortran, released under the GPL 3+ license, can be obtained with

```
sudo apt-get install gfortran
```

or one could build from source by visiting

<http://gcc.gnu.org/wiki/GFortran>.

3.1.3 CMake configuration variables

See `initial_cache.cmake.example` in the root directory for a list of configuration variables. You can copy and work with that file directly. There is thus no need to list them separately here. We only mention the most important one here, `BUILD_CXX_VERSION`, which determines whether the C++ version or the Fortran version of GADfit is built. Default is to build the C++ version.

3.2 Runtime linking

This section demonstrates how to compile and run a C++/Fortran program with calls to GADfit. Assuming we have successfully compiled GADfit as a shared library and created an input file `main.cpp/main.f90`, the following steps, if executed from the working directory, should successfully compile and execute. How to write an input file is explained in Sec. 4.2.

Note that you are free to use a different build system or build system generator, write your own makefile, or issue everything from the command line. However, those approaches tend to be more system specific. Here we are only focusing on how to include GADfit in CMake projects.

3.2.1 C++

Create a file called `CMakeLists.txt` with the following contents:

```
cmake_minimum_required(VERSION 3.13)
project(run LANGUAGES CXX)
add_executable(main main.cpp)
# If spdlog is installed to a non-standard location, specify it
# here. If installed to a standard system location or not
# installed at all, no need to do anything here.
set(spdlog_dir /usr/local/spdlog)
find_package(spdlog QUIET HINTS ${spdlog_dir})
# Directory where GADfit is installed. Can also point to the
# build directory.
set(gadfit_dir /usr/local/gadfit)
find_package(gadfit HINTS ${gadfit_dir})
target_link_libraries(main PUBLIC gadfit::gadfit)
```

Edit this file to reflect your environment. Next, create a build directory, configure and compile:

```
mkdir build && cd build
cmake ..
cmake --build .
```

The executable is produced in the same directory so you can run it with

```
export OMP_NUM_THREADS=8
./main
```

If the C++ compiler does not have OpenMP support the export statement has no effect.

3.2.2 Fortran

Create a file called `CMakeLists.txt` with the following contents:

```
cmake_minimum_required(VERSION 3.13)
project(run LANGUAGES Fortran)
add_executable(main main.F90)
# Directory where GADfit is installed. Can also point to the
# build directory.
set(gadfit_dir /usr/local/gadfit)
find_package(gadfit HINTS ${gadfit_dir})
# Uncomment the MKL libraries or specify your own
target_link_libraries(main PUBLIC
    gadfit::gadfit
    # /opt/intel/mkl/lib/intel64/libmkl_intel_lp64.so
    # /opt/intel/mkl/lib/intel64/libmkl_sequential.so
    # /opt/intel/mkl/lib/intel64/libmkl_core.so
)
```

Edit this file to reflect your environment. Next, create a build directory, configure and compile:

```
mkdir build && cd build
# cmake -DCMAKE_BUILD_TYPE=Release ..
# cmake -DCMAKE_Fortran_FLAGS_RELEASE="-fcoarray=lib" ..
cmake ..
cmake --build .
```

If GADfit was compiled with multi-image support, uncomment the two lines above. The executable is produced in the same directory so you can run it with

```
./main
```

With the multi-image version the number of images is set through the environment,

```
GFORTTRAN_NUM_IMAGES=2 ./main
```

(`FOR_COARRAY_NUM_IMAGES` with the Intel compiler).

For legacy reasons, we also include here a shell script for compiling and running a Fortran program with calls to GADfit.

```
#!/bin/bash
```

```
GAD=/usr/local/gadfit
```

```

# GNU
FC=gfortran
FFLAGS="-fcoarray=lib -J$GAD/include -L$GAD/lib"
LIBS="$LIBS -lcac_shmem"
# Intel
#FC=ifort
#FFLAGS="-coarray=shared -I$GAD/include -L$GAD/lib"

# Lapack
LIBS="$LIBS -llapack"

# Atlas
#FFLAGS="$FFLAGS -L/usr/local/atlas/lib"
#LIBS="$LIBS -ltatlas"

# MKL (for Intel and GNU compilers)
#MKLROOT=/opt/intel/mkl
#FFLAGS="$FFLAGS -L$MKLROOT/lib/intel64"
#export LD_LIBRARY_PATH=$MKLROOT/lib/intel64:$LD_LIBRARY_PATH
#LIBS="$LIBS -lmkl_intel_lp64 -lmkl_sequential -lmkl_core"
#LIBS="$LIBS -lmkl_gf_lp64 -lmkl_sequential -lmkl_core -lpthread"

export LD_LIBRARY_PATH=$GAD/lib:$LD_LIBRARY_PATH
$FC $FFLAGS main.f90 -o main -lgadfit $LIBS && \
GFORTRAN_NUM_IMAGES=4 ./main # GNU
#./main # Intel

```

This may need to be modified to reflect the user's environment. The Coarray runtime library must be the same one that GADfit was built against (see Sec. 4.1.2); with `caf_single` replace `-lcac_shmem` by `-lcac_single`. If GADfit was built without `-fcoarray=lib`, no Coarray runtime library need be linked.

3.3 Notes on using GADfit in other CMake projects

If you have built and installed GADfit and want to use it in a CMake project, only two CMake statements are necessary as demonstrated in the above examples. Assuming that GADfit has been installed at a standard system location and the target name you want to link against GADfit is `myproject`, include the lines

```

find_package(gadfit)
target_link_libraries(myproject PRIVATE gadfit::gadfit)

```

in that project. If GADfit is not installed in a standard location, use the `PATHS` or `HINTS` argument to `find_package`, e.g.,

```

find_package(gadfit PATHS $ENV{HOME}/libs/gadfit)

```

Additionally, if you need to ensure that GADfit is not older than a specific version, use

```
find_package(gadfit 2.7 PATHS $ENV{HOME}/libs/gadfit)
```

In this example, if the version of the installed copy is less than 2.7, CMake will exit with an error. Any version equal to or greater than 2.7 is fine. In order to see all the options of `find_package`, run

```
cmake --help find_package
```

or consult the online documentation.

If you want to link against GADfit without installing it first, the `PATHS` argument can point directly to the GADfit build directory:

```
find_package(gadfit PATHS $ENV{HOME}/builds/gadfit)
```

This is useful for developers.

4 Usage

4.1 Parallelism

4.1.1 C++

The C++ implementation of GADfit is thread-safe and is parallelized using OpenMP. That means you can speed up a single instance of the solver and/or call the solver from multiple threads in an OpenMP parallel region. The number of threads working on a single instance are set by a user parameter (`LMSolver::settings::n_threads`) which defaults to 1. Regardless of the `OMP_NUM_THREADS` environment variable or a call to `omp_set_num_threads`, the fitting procedure still runs on a single thread unless the parameter is set.

Under normal usage no OpenMP directives or functions need to be explicitly called. However, when calling some of the lower levels functions directly, such as allocating the AD tape variables, they need to be accompanied by the OpenMP parallel directive. If unsure have a look at the tests or the source code to see how certain GADfit functions must be called.

4.1.2 Fortran

The Fortran version of GADfit is parallelized using Coarrays, a syntactic extension to the language that was included in the Fortran 2008 standard. Coarrays follow the SPMD parallelization scheme, where the main program is replicated at the start of execution. Each instance, called an image, has its own set of private variables plus so-called co-variables that are addressable by other images. Coarrays allow to parallelize programs with little effort compared to MPI or OpenMP, although it is often either MPI or OpenMP that runs under the hood.

In order to compile a Coarray program with GFortran, the compiler flag `-fcoarray=lib` must be used and a Coarray runtime library must be linked. GFortran ships two such libraries: `libcaf_single`, which supports a single image only, and, since GCC 15, `libcaf_shmem`, where the images are separate processes communicating through shared memory. Which one is used is determined by the `CAF_LIBRARY` configuration variable,

```
cmake -D CAF_LIBRARY=caf_shmem ..
```

If it is left unset, `caf_single` is used. The number of images is set with the `GFORTTRAN_NUM_IMAGES` environment variable, which by default equals the number of cores,

```
GFORTTRAN_NUM_IMAGES=4 ./main
```

The same applies to the tests, which are launched through the `TESTS_RUN_PREFIX` variable, e.g.

```
-D TESTS_RUN_PREFIX="env;GFORTTRAN_NUM_IMAGES=4".
```

4.2 Example input

Suppose we have made two measurements of a decay process, which are to be fitted against

$$I(t) = I_0 e^{-t/\tau} + bgr. \quad (42)$$

The decay curves correspond to the same physical phenomenon, meaning they share the same decay time τ , but have been performed in slightly different experimental conditions with different initial amplitudes I_0 and backgrounds bgr . To get the best estimate for the fitting parameters, the curves should be fitted simultaneously with τ as a global fitting parameter and I_{01} , I_{02} , bgr_1 , and bgr_2 as local parameters.

4.2.1 C++

An input file `example.cpp` that solves the nonlinear least squares problem is found in `~gadfit/c++/tests`. Run it by invoking

```
cmake --build . --target example
```

4.2.2 Fortran

An input file `example.F90` that solves the nonlinear least squares problem is found in `~gadfit/fortran/tests`. Run it by invoking

```
cmake --build . --target example
```

The data files `example_data1` and `example_data2` are required to contain at least two columns (the rest are ignored). Any line beginning with a non-number is treated as a comment. In the following, we have chosen to put the fitting function into a Fortran module, followed by the main program which contains

a small set of commands to perform the fitting procedure. Users with more knowledge of Fortran may wish to do it differently. We shall first describe how to build the fitting function.

4.3 Defining the fitting function

4.3.1 C++

Here is the translation of the fitting function defined by Eq. (42):

```
static auto exponential(
    const std::vector<gadfit::AdVar>& parameters,
    const double x) -> gadfit::AdVar
{
    return parameters[0] * exp(-x / parameters[1])
        + parameters[2];
}
```

It takes a parameter vector and an argument at which to evaluate the function. Definition of the function body is very simple. It reads just like a normal statement using floating point variables, except that `parameters` is a vector of AD variables. One may also use aliases for the parameter names to make it a bit more readable (see `example.cpp`).

4.3.2 Fortran

In the Fortran version, all user defined functions are derived from an abstract type `fitfunc` (making them *derived classes* in OOP terms). Two procedures must always be defined: a constructor, which specifies the number of fitting parameters, and the function body. While the user is advised to work through a full Fortran tutorial, in the following some effort has been made to explain the peculiarities of the language.

```
module test_f
  ! This module defines the function I(x) = I0*exp(-t/tau)+bgr.
  ! In Fortran, comments begin with an "!".

  use ad ! Imports the module containing the procedures for AD.
          ! Similar to the preprocessor #include directive.
  use fitfunction ! Imports the abstract class for the fitting
                  ! function.
  use gadf_constants ! Imports the floating point precision
                    ! specifier (kp).

  implicit none ! This should be present in all modules.

  ! We have decided to call the type of the new fitting function
  ! 'exponential'. init and eval are the constructor and the
```

```

! function body, which must be renamed and implemented below.
type, extends(fitfunc) :: exponential
contains
  procedure :: init => init_exponential
  procedure :: eval => eval_exponential
end type exponential

contains
! The interfaces of the following procedures, i.e., the type of
! the arguments and of the return value, are fixed.
subroutine init_exponential(this)
  class(exponential), intent(out) :: this
  ! Allocate memory to the fitting parameter array.
  allocate(this%pars(3))
  ! In C++, this would be something like
  ! this->pars = new advar[3];
end subroutine init_exponential

type(advar) function eval_exponential(this, x) result(y)
  class(exponential), intent(in) :: this
  real(kp), intent(in) :: x
  y = this%pars(1)*exp(-x/this%pars(2)) + this%pars(3)
  ! In this context, I0 = this%pars(1), tau = this%pars(2), and
  ! bgr = this%pars(3).
end function eval_exponential
end module test_f

```

This might seem complicated at first, but have a look at `~gadfit/fortran/tests/function_template`, which is a stripped-down version of the above. The angle-bracketed words need to be replaced by user-defined values and the only effort lies in the description of the function body. In order to test whether the function has been correctly defined, one can use the same code as in the next section, but comment out the call to `gadf_fit`. If there are no data sets, then `gadf_print` must be explicitly supplied with the arguments `begin` and `end`, which define the argument range.

The above is a very simple example highlighting only some of the functionality. For more complex examples, see the tests in `~gadfit/fortran/tests`.

As a comment, we have used the module approach because the user might wish in the future to put the most commonly used fitting functions into a single module that can then be conveniently included, e.g., as a library in any source file. This is just a suggestion.

4.4 Fitting procedure

4.4.1 C++

First, initialize the Levenberg-Marquardt solver,

```
gadfit::LMSolver solver { f };
```

where `f` is any function with the correct signature, e.g. `exponential` from Sec. 4.3.1. Next, add any number of data sets,

```
solver.addDataset(x_data_1, y_data_1);
solver.addDataset(x_data_2, y_data_2);
```

If you look into `lm_solver_data.h` you'll see that the input data are of type `std::array`. Internally the data are captured by `std::span` which is a view of the data and thus avoids a copy. The data could also be of type `std::vector` or anything that is compatible with `std::span`. It is up to you to ensure that the data doesn't go out of scope before the solver is run. Then set the initial values of all fitting parameters. In this example, all starting values are set to 1.0.

```
solver.setPar(0, 1.0, true, 0); // I0 of curve 1
solver.setPar(2, 1.0, true, 0); // bgr of curve 1
solver.setPar(0, 1.0, true, 1); // I0 of curve 2
solver.setPar(2, 1.0, true, 1); // bgr of curve 2
solver.setPar(1, 1.0, true);    // tau
```

Here the numeration of parameters is 0: I_0 , 1: τ , 2: bgr. This is determined by how the body of the fitting function is defined (see Sec. 4.3.1). Finally, call the fitting procedure:

```
solver.fit();
```

4.4.2 Fortran

Here are the commands necessary to run the example. For a complete overview of input variables, see Sec. 4.5.2.

```
use test_f ! Include the above-defined fitting function
use gadfit ! and the main library.

implicit none

type(exponential) :: f ! An instance of the fitting function

! Initialize GADfit with the fitting function and the number of
! data sets.
call gadf_init(f, 2)

! Include both decay curves. The argument must be full or
! relative path to the data. TESTS_BLD is expanded by the
! preprocessor to ~build/fortran/tests.
call gadf_add_dataset(TESTS_BLD//'/example_data1')
call gadf_add_dataset(TESTS_BLD//'/example_data2')
```

```

! The initial guess for all fitting parameters is 1.0. The
! first argument denotes the data set, the second argument the
! parameter, third argument its value, and fourth whether the
! parameter is allowed to vary or is kept fixed.
call gadf_set(1, 1, 1.0, .true.) ! I01
call gadf_set(2, 1, 1.0, .true.) ! I02
call gadf_set(1, 3, 1.0, .true.) ! bgr1
call gadf_set(2, 3, 1.0, .true.) ! bgr2
! Global parameters don't have the data set argument.
call gadf_set(2, 1.0, .true.) ! tau

! The uncertainties of the data points determine their
! weighting in the fitting procedure. Here we are assuming shot
! noise, i.e., the error of each data point is proportional to
! the square root of its value. Default is no weighting.
call gadf_set_errors(SQRT_Y)

! Perform the fitting procedure starting with lambda=10. If the
! procedure doesn't converge, we should restart with a higher
! value or modify any of the other arguments to gadf_fit. All
! the arguments are optional with reasonable default values.
call gadf_fit(lambda=10.0)

! The results are saved into ~gadfit/fortran/tests
call gadf_print(output=TESTS_BLD//'/example_results')
call gadf_close() ! Free memory
end program

```

In order to run the above code, one can manually link and compile from the command line, use a small script like in Sec. 3.2, or simply issue

```
cmake --build . --target example
```

from the build directory. If no linear algebra library has been specified, it will take a moment to build one.

The results of the calculation will be printed to the standard output and also to `~build/fortran/tests/example_results_parameters`. The decay time should have the value of about 20.5. If the errors are the true experimental uncertainties, then for a good fit the reduced sum of squares should be close to 1. The results can be visualized by opening gnuplot in the directory `~build/fortran/tests` and issuing

```

p 'example_data1' t 'data1', 'example_data2' t 'data2', \
'example_results' u 1:2 w l t 'fit1', '' u 1:3 w l t 'fit2'

```

The rest of the examples in `~gadfit/fortran/tests` can be built and run with

```
cmake --build . --target test
```

The fitting functions used in the tests are

$$f_1(x) = f_{\max} e^{-(x-x_0)^2/a^2} + bgr,$$

$$f_2(x) = \pi \int_0^x t^a e^{-bt^2} dt,$$

$$f(x) = \int_0^\infty dt \int_0^{x/b} dy \frac{\ln[(e^y - 1)(1 + a \operatorname{erf} t) + 1]}{xy} e^{-t},$$

where erf is the error function.

4.5 User interface

4.5.1 C++

You will mainly be interacting with the `gadfit::LMSolver` class. Here is an overview of the most important member functions and variables. Several of the functions and variables come with default values. Those are easily found in the source code (`c++/gadfit/lm_solver.h`) and not duplicated here. The following functions and variables should be called on an `LMSolver` instance (i.e. not directly like static functions and variables).

```
gadfit::LMSolver::LMSolver(const fitSignature& function_body)
```

- `function_body`: the function body. See Sec. 4.3.1 for how to define one.

```
gadfit::LMSolver::addDataset(const T& x_data,
                             const T& y_data,
                             const T& errors = {})
```

Add a dataset. The type `T` needs to be compatible with `std::span`, e.g. `std::vector<double>`.

- `x_data`: array of independent variables of a data set.
- `y_data`: array of dependent variables of a data set.
- `errors`: if present, data point errors (standard deviations). Default is to set all errors equal to 1, i.e. all data points have the same weight.

```
gadfit::LMSolver::addDataset(const int n_datapoints,
                             const double* x_data,
                             const double* y_data,
                             const double* errors = nullptr)
```

A low level version of `addDataset` that deals directly with pointers. Use this if pointing to a segment of an array or when the template type `T` above is incompatible with `std::span`.

- `x_data`: shared pointer of an array of independent variables of a data set.
- `y_data`: shared pointer of an array of dependent variables of a data set.
- `errors`: if present, defines a shared pointer of data point errors. Default is to set all errors equal to 1, i.e. all data points have the same weight.

```
gadfit::LMSolver::setPar(
    const int i_par,
    const double val,
    const bool active = false,
    const int i_dataset = global_dataset_idx,
    const std::string& parameter_name = "")
```

Set fitting parameters.

- `i_par`: an index of the parameter array as defined in the fitting function.
- `val`: initial value of a fitting parameter.
- `active`: whether this is an active or a passive (constant) fitting parameter.
- `i_dataset`: if present, specifies a data set. If not present, defines a global fitting parameter.
- `parameter_name`: if present, displays the parameter name during fitting procedure. Default is to display the parameter index.

```
gadfit::LMSolver::setPar(const int i_par,
                        const double val,
                        const bool active,
                        const std::string& parameter_name)
```

Call this for setting a global parameter with a name. Otherwise it is the same as the other `setPar`.

```
gadfit::LMSolver::settings
```

A struct containing parameters that define runtime behavior. Find the default values in `c++/gadfit/lm_solver.h`.

- `iteration_limit (int)`: iteration limit.
- `lambda_down (double)`: Factor by which λ is increased for accepted steps.
- `lambda_up (double)`: Factor by which λ is decreased for rejected steps.
- `lambda_incs (int)`: Number of times λ is allowed to increase consecutively without terminating the fitting procedure.

- **damp_max** (bool): Whether to update the damping matrix with the largest entries of $J^T J$ yet encountered. If **false**, the classical Marquardt scheme is used.
- **DTD_min** (std::vector<double>): Minimum initial values of the diagonal entries of the damping matrix. Only relevant if **damp_max == true**. Length of this array must equal the number of fitting parameters.
- **acceleration_threshold** (double): Acceleration threshold [α in Eq. (7)]. If negative, the acceleration term is not calculated.
- **loss**: Type of loss function:
 - **linear**: $\rho(z) = z$ where z is the squared residual (default)
 - **cauchy**: $\rho(z) = \ln(1 + z)$
 - **huber**: $\rho(z) = z$ if $z \leq 1$ and $\rho(z) = 2(\sqrt{z} - 1)$ otherwise
- **n_threads**: Number of OpenMP threads. Default is 1.

`gadfit::LMSolver::settings::verbosity`

Flags for controlling output. Usage follows the standard rules of bit manipulation in C++. For example, for turning on the **delta1** and **delta2** flags,

```
gadfit::LMSolver solver { f };
f.settings.verbosity = gadfit::io::delta1 | gadfit::io::delta2;
```

The following flags are supported.

- **delta1**: Show the change in parameter value after each iteration.
- **delta2**: Show the acceleration term of each parameter after each iteration. Note that the actual contribution to the update vector is $-\frac{1}{2}\delta_2$. Only shown if **acceleration_threshold** is also set.
- **timings**: Show detailed timing information for all processes.
- **hide_local**: Do not show local parameters during the fitting procedure.
- **hide_global**: Do not show global parameters during the fitting procedure.
- **hide_all**: Do not show anything.
- **final_only**: Only show results after the final iteration.
- **all**: Turn on all flags.

`gadfit::LMSolver::fit(double lambda = default_lambda)`

Performs the fitting procedure.

- `lambda`: the adaptive damping parameter.

```
gadfit::LMsolver::getParValue(const int i_par,
                              const int i_dataset = 0)
```

Get parameter `i_par` corresponding to data set `i_dataset`.

```
gadfit::LMsolver::getValue(const double arg,
                           const int i_dataset = 0)
```

Get the fitting function value at `arg` using parameters corresponding to data set `i_dataset`.

```
gadfit::LMsolver::getJacobian()
gadfit::LMsolver::getJTJ()
gadfit::LMsolver::getDTD()
gadfit::LMsolver::getLeftSide()
gadfit::LMsolver::getRightSide()
gadfit::LMsolver::getResiduals()
gadfit::LMsolver::getInvJTJ()
```

Get J , $J^T J$, $D^T D$, left side of Eq. (2), right side of Eq. (2), the residuals $y - f(\beta)$, or $(J^T J)^{-1}$, respectively.

```
gadfit::initIntegration(const int workspace_size = 1000,
                       const int n_workspaces = 1)
```

Allocate memory for integration routines. This must be called before `LMsolver::fit` if the fitting function contains integrals.

- `workspace_size`: number of Gauss-Kronrod sub-intervals.
- `n_workspaces`: number of integration levels (1 for single integrals, 2 for double integrals).

```
gadfit::freeIntegration()
```

Deallocate integration workspaces.

```
spdlog::set_pattern(pattern_string)
```

GADfit makes of `spdlog` for writing output. See the `spdlog` project's homepage for full documentation. Here we only describe functions that are most relevant to GADfit.

- `pattern_string`: format for writing output. For instance, if the default output is too verbose, include something like `spdlog::set_pattern("%v");` before the first call to `gadfit::LMsolver::fit`.

```
spdlog::set_level(spdlog::level)
```

Set global log level

- `spdlog::level`: logging levels.
 - `spdlog::level::info`: normal output.
 - `spdlog::level::debug`: more verbose output.
 - `spdlog::level::off`: turn off all output.

4.5.2 Fortran

Before using GADfit, here are a some general remarks about argument processing in a Fortran.

- There is no automatic argument type conversion when calling a procedure. If the procedure expects a double precision number, then this is exactly what the user must supply. In C terms, if the function expects a 'double', then a 'float' will cause an error. While it can be compiler dependent, numbers such as 3.2 or 5e7 are generally interpreted as single precision. In GADfit, higher precision can be obtained by appending numbers with the "kind parameter" (3.2_kp, 5e7_kp) in which case they become double or quadruple precision, depending on how GADfit was configured. With double precision, instead of _kp we can also use the d descriptor (e.g. 3.2d0).

Internally, most procedures use at least double precision. However, for user convenience, some procedures are also available in single precision. For example, instead of

```
call gadf_set(1, 3.2_kp)
```

we can also have

```
call gadf_set(1, 3.2)
```

Single precision is not that important when setting initial parameter values. (The decimal point or an exponent must still be present, else the argument is interpreted as an integer.)

- Procedure arguments can be given as a value (`call gadf_fit(1.0)`) or as a name-value pair (`call gadf_fit(lambda=1.0)`). When using the latter option, the arguments can be given in any order. It is also useful when calling a procedure that has many optional arguments but the user wishes to provide only some of them.

The next section contains a detailed list of all procedures that are required for normal use. The procedure arguments are given according to the format `<type-specifier>::<variable>`, where the type-specifier can be `real(kp)` – double or quad precision real number; `real(real32)` – single precision real

number; **integer** – integer; **character(*)** – character array ('asdf'); **logical** – truth value, can be either **.true.** or **.false.**.. If there is more than one allowed type for the argument, the choices are separated by a slash. Optional arguments have the keyword **optional**. All procedures are subroutines except **integrate**, which is a function.

```
gadf_init(f, num_datasets, ad_memory, sweep_size, trace_size,
          const_size, rel_error, rel_error_outer, ws_size,
          ws_size_inner, integration_rule)
```

Initializes the workspace. This procedure must always be called when performing a fitting procedure.

- **class(fitfunc) :: f**
The fitting function. **class(fitfunc)** denotes a polymorphic variable. In practice, **f** has an explicit type like **type(exponential)** in the example.
- **integer, optional :: num_datasets**
Number of data sets.
Default: 1.
- **character(*), optional :: ad_memory**
Allocates memory for the work variables of the AD reverse mode so that **forward_values** and **adjoints** contain x , **trace** $4x$, and **ad_constants** $\frac{1}{2}x$ elements, where x is such that the total amount of memory reserved is equal to that given by **ad_memory**, whose format is “number unit”, where “unit” is B, kB, MB, or GB (also acceptable are b, kb, mb, or gb). Example: **ad_memory='1.25 MB'**. Overrides **sweep_size**, **trace_size**, and **const_size**, which can be used for fine-tuning memory allocation.
Default: none.
- **integer, optional :: sweep_size**
Size of **forward_values** and **adjoints**.
Default: 10 000.
- **integer, optional :: trace_size**
Size of **trace**.
Default: $4 \times 10\,000$.
- **integer, optional :: const_size**
Size of **ad_constants**.
Default: $\frac{1}{2} \times 10\,000$.

- `real(kp), optional :: rel_error`
Tolerance for the relative error of numerical integration. When dealing with double integrals, this applies to the outer integral.
Default (single integrals): $100 \times \text{machine epsilon}$ or
Default (double integrals): $1000 \times \text{machine epsilon}$
- `real(kp), optional :: rel_error_inner`
Tolerance for the relative error of the inner integral.
Default: $100 \times \text{machine epsilon}$.
- `integer, optional :: ws_size`
Size of the integration workspace. With double integrals this refers to the outer integral. It is assumed that there are at most this many integration subintervals.
Default: 1000.
- `integer, optional :: ws_size_inner`
Size of the integration workspace for the inner integral when using double integrals.
Default: 1000.
- `integer, optional :: integration_rule`
Sets the Gauss-Kronrod integration rule. Allowed values are
GAUSS_KRONROD_15P, GAUSS_KRONROD_21P, GAUSS_KRONROD_31P,
GAUSS_KRONROD_41P, GAUSS_KRONROD_51P, and GAUSS_KRONROD_61P.
Default: GAUSS_KRONROD_15P.

`gadf_add_dataset(path)`

Reads a data set. This procedure must be called as many times as there are data sets, which is determined by the `num_datasets` argument to `gadf_init`.

- `character(*) :: path`
Full or relative path to the data file.

`gadf_add_dataset(x_data, y_data, weights)`

Reads a data set. This procedure must be called as many times as there are data sets, which is determined by the `num_datasets` argument to `gadf_init`.

- `real(kp) :: x_data(:)`
Array containing the x-values (data point coordinates).
- `real(kp) :: y_data(:)`
Array containing the y-values (data point values).

- `real(kp), optional :: weights(:)`
Array containig the data point weights. Only used with the `USER` argument to `gadf_set_errors` (see below).

`gadf_set(dataset_i, par, value, active)`

Defines the parameter as local, sets its value, and marks it either active or passive.

- `integer :: dataset_i`
The data set index.
- `integer/character(*) :: par`
Either the parameter index or its name.
- `real(real32)/real(kp) :: val`
Parameter value in either single or higher precision.
- `logical, optional :: active`
If `.true.` the parameter is active, else passive. In the current implementation, if a parameter is local it is either active or passive for all data sets.
Default: `.false..`

`gadf_set(par, value, active)`

Similar to the other `gadf_set` except that this one defines either a global fitting parameter or a global constant. If only one data set is used, it doesn't matter which procedure is used.

`gadf_set_verbosity(scope, digits, timing, memory, workloads,
 delta1, delta2, cos_phi, grad_chi2, uphill,
 acc, output)`

Gives the user some control over how the results are displayed. All flags can be set to `.true..` For instance, one might wish know $|\cos \phi|$ of Eq. (20) but not used it as a convergence criterion.

- `integer, optional :: scope`
Whether to show only local or global parameters or both during the fitting procedure. Allowed values are `GLOBAL`, `LOCAL`, and `GLOBAL_AND_LOCAL`.
Default: `GLOBAL_AND_LOCAL`.
- `integer, optional :: digits`
How many significant digits of the fitting parameters are shown during the fitting procedure.
Default: 7.

- `logical, optional :: timing`

Whether to show the timing summary after the fitting procedure. The cpu and wall times are measured for $\mathbf{J}$, χ^2 , $\mathbf{\Omega}$, and simple linear algebra operations ($\mathbf{J}^T$, $\mathbf{J}^T \mathbf{J}$, $\mathbf{J}^T(\mathbf{y} - \mathbf{f})$, ...). The cost of calculating $\mathbf{J}$ also includes the residual vector $\mathbf{y} - \mathbf{f}$, which is produced as part of the forward sweep. All other parts of the fitting procedure are serial. Since the serial portion of the code is negligible, all images should in principle do the same amount of work. The timing summary thus contains information about the load imbalance. It contains the following parts:

- The cpu times per call are averaged over images and over the number of calls to the procedure.
- The relative cost is the cpu time averaged over images and divided by the total cpu time spent in the main loop. With perfect scaling the relative costs should add up to 100%.
- Detailed timing is shown for $\mathbf{J}$, χ^2 , and $\mathbf{\Omega}$. The reported total cpu and wall times are over all serial and parallel parts of the main loop, and do not correspond exactly to the sum of the shown quantities.

Default: `.false..`

- `logical, optional :: memory`

Whether to show memory usage after the fitting procedure. Only the peak usage is shown, which is not necessarily representative of the actual load.

Default: `.false..`

- `logical, optional :: workloads`

Whether to show the workload of each image. If the loads are very uneven, it is worth considering using load balancing.

Default: `.false..`

- `logical, optional :: delta1`

Whether to show δ_1 , the velocity term.

Default: `.false..`

- `logical, optional :: delta2`

Whether to show δ_2 , the acceleration term.

Default: `.false..`

- `logical, optional :: cos_phi`

Whether to show $|\cos \phi|$ [Eq. (20)].

Default: `.false..`

- `logical, optional :: grad_chi2`
Whether to show $|\nabla\chi^2| = |2\mathbf{J}^T(\mathbf{y} - \mathbf{f})|$.
Default: `.false..`
- `logical, optional :: uphill`
Whether to show $\cos(\delta_i, \delta_{i-1})$ [Eq. (8)].
Default: `.false..`
- `logical, optional :: acc`
Whether to show the ratio of the contributions from the velocity and the acceleration terms [Eq. (7)].
Default: `.false..`
- `character(*), optional :: output`
Where to send the output. Can be a file or, in general, any I/O device. On Linux, using `'/dev/null'` suppresses all output except errors and warnings, which are sent to stderr.
Default: stdout (standard output).

`gadf_set_errors(e)`

Specifies the data point errors – σ_i in Eq. (3). If this procedure is not called, the errors are set to 1.

- `integer, optional :: e`
Allowed values are `NONE` – $\sigma_i = 1$ (the default); `SQRT_Y` – $\sigma_i = \sqrt{y_i}$ (shot noise); `PROPTO_Y` – $\sigma_i = y_i$; `INVERSE_Y` – $\sigma_i = 1/y_i$; `USER` – user defined uncertainties, must be supplied as the 3rd column in the data files or as a separate array.

`gadf_fit(lambda, lam_up, lam_down, accth, grad_chi2, cos_phi, rel_error, rel_error_global, chi2_rel, chi2_abs, lam_incs, uphill, max_iter, damp_max, nielsen, umnigh, load_balancing, use_ad)`

Performs the fitting procedure. The procedure stops if any of the convergence criteria is satisfied.

- `real(real32), optional :: lambda`
The adaptive damping parameter.
Default: 1.
- `real(real32), optional :: lam_up`
Factor by which λ is increased for accepted steps.
Default: 10.

- `real(real32), optional :: lam_down`
Factor by which λ is decreased for rejected steps.
Default: 10.
- `real(real32), optional :: accth`
Acceleration threshold [α in Eq. (7)]. If zero, the acceleration term is not calculated.
Default: 0.
- `real(real32), optional :: grad_chi2`
Tolerance for $|\nabla\chi^2| = |2\mathbf{J}^T(\mathbf{y} - \mathbf{f})|$.
Default: 0.
- `real(real32), optional :: cos_phi`
Tolerance for the cosine of the angle between the residual vector and the range of the Jacobian [Eq. (20)].
Default: 0.
- `real(real32), optional :: rel_error`
Tolerance for the relative change in any fitting parameter for two consecutive iterations.
Default: 0.
- `real(real32), optional :: rel_error_global`
Same as `rel_error` but applies only to global parameters.
Default: 0.
- `real(real32), optional :: chi2_rel`
Tolerance for the relative change in χ^2 for two consecutive iterations.
Default: 0.
- `real(real32), optional :: chi2_abs`
Tolerance for the reduced sum of squares.
Default: 0.
- `real(kp), optional :: DTD_min(:)`
Minimum values of the diagonal entries of the damping matrix.
Default: 0.
- `integer, optional :: lam_incs`
Number of times λ is allowed to increase consecutively without terminating the fitting procedure.
Default: 2.

- `integer, optional :: uphill`
The exponent b in Eq. (8). If zero, uphill steps are not allowed.
Default: 0.
- `integer, optional :: max_iter`
Iteration limit.
Default: unlimited.
- `logical, optional :: damp_max`
Whether to update the damping matrix with the largest entries of $J^T J$ yet encountered. If `.false.`, the classical Marquardt scheme is used.
Default: `.true.`.
- `logical, optional :: nielsen`
Whether to update λ with $\max[1/\lambda_{\downarrow}, 1 - (2\rho - 1)^3]$. See the paragraph on updating λ in Sec. 2.1. ρ includes only the velocity term.
Default: `.false.`.
- `logical, optional :: umnigh`
Whether to update λ according to Umrigar and Nightingale.
Default: `.false.`.
- `logical, optional :: load_balancing`
Whether to use load balancing. If `.true.`, the input data is redistributed among the images in an effort to achieve equal workloads. The redistribution is based on the workloads of the previous iteration and is repeated after each iteration. Load balancing makes sense with integrals and other constructs whose cost heavily depends on the argument.
Default: `.false.`.
- `logical, optional :: use_ad`
Whether to use automatic differentiation for the computation of the Jacobian (both the velocity and acceleration terms). If false, use finite differences.
Default: `.true.`.

```
gadf_print(begin, end, points, output, grouped, logplot,
           begin_kp, end_kp)
```

Prints the results to an I/O device. The results include the theoretical curve(s), fitting parameter values, and summary of resources used. If no fitting procedure has been performed, only the theoretical curve is printed.

- `real(real32), optional :: begin`
The lower bound for the x -values.
Default: taken from the input data.
- `real(real32), optional :: end`
The upper bound for the x -values.
Default: taken from the input data.
- `integer, optional :: points`
Number of points printed per curve.
Default: 200.
- `character(*), optional :: output`
Output file root name.
Default: 'out'.
- `logical, optional :: grouped`
Whether to group all curves into a single file.
Default: `.true..`
- `logical, optional :: logplot`
Whether to produce results suitable for a $(\log x, y)$ -plot.
Default: `.false..`
- `real(kp), optional :: begin_kp`
Same as `begin` but with double/quad precision; overrides `begin`.
Default: none.
- `real(kp), optional :: end_kp`
Same as `end` but with double/quad precision; overrides `end`.
Default: none.

`gadf_close()`

Frees all work variables. After this, it is safe to call `gadf_init` again.

`integrate(f, pars, lower, upper, rel_error, abs_error)`

Performs numerical integration. Can be used as part of a fitting function. The result is of type `advar`.

- `type(advar) function f(x, pars)`
 `real(kp), intent(in) :: x`
 `type(advar), intent(in out) :: pars(:)`
`end function f`
The integrand.

- `type(advar) :: pars(:)`
Parameters passed to the integrand.
- `type(advar)/real(kp)/integer :: lower`
Lower integration bound. Can be an active/passive fitting parameter, a real number, `-INFINITY`, or `INFINITY`.
- `type(advar)/real(kp)/integer :: upper`
Upper integration bound. Same comments as with `lower`.
- `real(kp), optional :: relative_error`
Tolerance for the relative error of the integral. Overrides `gadf_init`.
Default: whatever was specified with `gadf_init` or 100 (1000) $\times$ machine epsilon for the inner (outer) integral.
- `real(kp), optional :: absolute_error`
Tolerance for the absolute error.
Default: 0.

4.6 Internals

4.6.1 Fortran

The previous sections were about the main commands that would normally be used. This section describes more advanced usage of GADfit for experimental, debugging, or other purposes. Some knowledge of Fortran is assumed.

Basic usage of the `fitfunc` class. As was mentioned earlier, the user can test the fitting function by calling `gadf_print` without performing a fitting procedure. It is, however, possible to work directly with the fitting function, which inherits from the `fitfunc` class. The following code snippet prints the value of a function with the argument 2.0. It is assumed that GADfit has been compiled with double precision.

```
type(test) :: f                ! 'test' extends fitfunc.
real(dp) :: dummy
!call init_integration()      ! if necessary or
!call init_integration_dbl() ! if necessary.
call f%init()
call f%set(1, 1d0)            ! Has just one parameter.
dummy = f%eval(2d0)           ! Converts AD variable to real.
write(*,*) dummy              ! Function value at 2.0.
call f%destroy()
call free_integration()       ! Optional but never hurts.
```

Here and elsewhere, a dummy variable must be used since a component of a function result of derived type cannot be directly referenced in Fortran.

If the only aim is to print the function value(s) at some point(s), then the code can be slightly reduced by using the procedures of Sec. 4.5.2. After calls to `gadf_init` and `gadf_set`, multiple instances of the function, each with an independent set of parameters, are copied to the `fitfuncs` array, which is protected (read-only public). The above code then reduces to

```
type(test) :: f
real(dp) :: dummy
call gadf_init(f)
call gadf_set(1, 1d0)
dummy = fitfuncs(1)%eval(2d0)
write(*,*) dummy
call gadf_close()
```

Another reason for directly working with the `fitfunc` class is the calculation of derivatives using finite differences, which can be used for testing new AD elemental operations. In the next snippet, the derivatives are calculated with respect to parameters 1 and 3, while parameter 2 is passive.

```
type(test) :: f
real(dp) :: grad(2)
call f%init()
call f%set(1, 1.2d0)
call f%set(2, -0.1d0)
call f%set(3, 1d17)
call f%grad_finite(2d0, [1,3], grad)
write(*,*) grad
call f%destroy()
```

In order to calculate the 2nd directional derivative in the (0.3,0.7) direction in the space spanned by parameters 1 and 3, the above code can be modified by inserting

```
write(*,*) f%dir_deriv_2nd_finite(2d0, [1,3], [0.3d0,0.7d0])
```

after the parameters have been initialized. See the documentation of `grad_finite` and `dir_deriv_2nd_finite` in `fitfunction.f90` for a better understanding.

Both finite differences and AD are always available regardless of the value of `USE_AD` during configuration. This option only determines which one is used during the fitting procedure.

Basic usage of the AD forward mode. In the forward mode, parameters are made active by modifying both their `index` and `d` fields. In the interest of compactness, here we shall set the parameter values directly instead of using the more safe `set` procedure.

```

type(test) :: f
type(advar) :: dummy
call f%init()
f%pars = [1.2d0, -0.1d0, 1d17]
f%pars%index = [1,0,0]
f%pars%d = [1,0,0]
dummy = f%eval(2d0)
write(*,*) dummy%d
f%pars%index = [0,0,1]
f%pars%d = [0,0,1]
dummy = f%eval(2d0)
write(*,*) dummy%d
call f%destroy()

```

In order to calculate the 1st and 2nd directional derivatives in the (0.3,0.7) direction, the above code becomes

```

type(test) :: f
type(advar) :: dummy
call f%init()
f%pars = [1.2d0, -0.1d0, 1d17]
f%pars%index = [1,0,1]
f%pars%d = [0.3,0.0,0.7]
dummy = f%eval(2d0)
write(*,*) 'First dir deriv:', dummy%d
write(*,*) 'Second dir deriv:', dummy%dd

```

Basic usage of the AD reverse mode. The reverse mode is a bit more difficult to use manually since the user has to be more acquainted with the internal work variables of `automatic_differentiation.f90`. In particular, it is necessary to set `index_count` to the number of active parameters (index of the next AD variable will be `index_count+1`), and to initialize the array of intermediate values. The gradient calculation with respect to parameters 1 and 3, as in the previous examples, can then be achieved as follows.

```

type(test) :: f
real(dp) :: dummy
call ad_init_reverse()
call f%init()
f%pars = [1.2d0, -0.1d0, 1d17]
f%pars%index = [1,0,2]
index_count = 2
forward_values(1:2) = [f%pars(1)%val, f%pars(3)%val]
dummy = f%eval(2d0)
!write(*, '(g0)') trace(:trace_count)
call ad_grad(2)
write(*,*) adjoints(1:2)

```

```

call f%destroy()
call ad_close()

```

Uncommenting the line after function evaluation shows what the execution trace looks like after the forward sweep. If the function contains an integral, additional calls to `init_integration` or `init_integration_dbl` and `free_integration` are required.

Numerical integration. It is also possible to use GADfit just for evaluating integrals, although if this is the only purpose, there are better tools out there. The following example shows the calculation of $\int_{0.5}^{\infty} e^{-x} dx$:

```

type(advar) :: dummy, pars(0)
call init_integration()
dummy = integrate(f, pars, 0.5d0, INFINITY)
print*, dummy%val
call free_integration()
contains
type(advar) function f(x, pars) result(y)
  real(kp), intent(in) :: x
  type(advar), intent(in out) :: pars(:)
  y = exp(-x)
end function f

```

Creation of new elemental operations. New elemental operations can be introduced in `automatic_differentiation.f90`. The main steps are:

- A new operation code is created.
- A module procedure is overloaded.
- In the reverse mode, the operation body is supposed to do the following: the function body is evaluated; the newly created result variable is given a unique index; the result of evaluation is saved in `forward_values`; the indices of all arguments, that of the result variable, and the operation code are saved in `trace`; any constants are saved in `ad_constants`; `index_count`, `trace_count`, and `const_count` are increased accordingly. In the forward mode, the implementation is straightforward. In either case, if the argument is a passive variable, only the function body is evaluated. For a better understanding, have a look at the implementation of any unary operation in the source code.
- In the reverse mode, the calculation of the adjoints is implemented in the return sweep (`ad_grad`).

5 Developer notes

5.1 C++

5.1.1 Unit tests

Unit tests are written using the Catch2 framework. Catch2 needs to be installed separately if not already present on your system. The tests can be run with

```
cmake --build . --target test
```

or you can run them manually one by one. Each test is an executable in `c++/tests` in your build directory.

5.1.2 Code linters

Before committing, you should run clang-tidy on the source code. CMake has built-in support for that so all you need to do is run

```
cmake -DCMAKE_CXX_CLANG_TIDY=clang-tidy .
```

in the build directory and recompile. You can keep this on but if it noticeably slows down compilation you might want to turn it off during development with

```
cmake -U CMAKE_CXX_CLANG_TIDY .
```

We also use the clang-format code linter. However, since CMake does not have built-in support for clang-format it is up to the developer to find a way to run clang-format on the source code. As an example, you may run this in the directory with the C++ source files:

```
find -regextype posix-extended -regex ".*\.(cpp|h)" -exec \
sh -c 'clang-format -style=file {} | diff -u {} -' \;
```

If the code is perfectly valid, there should be not output. Otherwise, you should resolve the diff. This requirement does not hold for the tests directory.

5.1.3 Code coverage

Code coverage can be inspected by enabling the `INCLUDE_COVERAGE` CMake variable. This only works with the GNU compiler and requires LCOV to be installed. The flags defined by `COVERAGE_FLAGS` are then appended to the compilation flags (the default “-coverage” should be fine). If you then recompile and run the tests, it shows you the overall coverage rate for lines and functions and produces a detailed HTML report that you can inspect. Here is an example of how to programmatically turn on the coverage flag, run the coverage, and then turn it off again:

```
cmake -DINCLUDE_COVERAGE=ON .
cmake --build . --target coverage
cmake -DINCLUDE_COVERAGE=OFF .
```

5.1.4 Coding rules

Follow the C++20 standard. Otherwise, there are no additional coding rules because clang-tidy and clang-format are already quite exhaustive. If those pass you're good to go.

5.2 Fortran

5.2.1 Coding rules

Programming principles

- **Standard-conforming code.** This project is committed to following the Fortran 2008 standard (F2008), which is a minor improvement over the 2003 standard, but a major improvement over the commonly used Fortran 90, making coding much more efficient. It is not possible to list here all the rules that the developer is expected to follow when writing modern Fortran. There is not always a unique answer anyway. Many rules are expected to be self-evident. Here we concentrate on rules specific to this project and that are not standard in general for modern Fortran. A good place to start is to examine an existing module in addition to reading this.
- **Exception handling.** Use `check_err` after allocate and open statements. `safe_close` is optional and depends on factors such as the amount of data written. For deallocation use `safe_deallocate` and, if necessary, define a new subroutine for the `safe_deallocate` interface. For reporting errors, warnings, and comments, use the appropriate procedures from the messaging module.
- **Compilation stage.** The code should compile without any warnings when using debug flags with GFortran and preferably also with Ifort, although the latter is not a requirement.
- **Preprocessor.** Use preprocessor macros only for actual preprocessing and not for something that is a short-hand notation for a Fortran statement. Excessive use of macros makes it difficult for source code indexing tools to navigate through the source code.

Format

- **Statement order.** A module should have the following format:

```
module <module_name>
<'use' statements in alphabetical order>
implicit none
private
protected :: <list>
public :: <list>
```



```

    <enumerators>
    <constants>
    <derived type definitions>
    <interfaces>
    <variables>
contains
    <procedures>
end module <module_name>

```

Protected variables should not be listed after the `public` statement, but declared public elsewhere. Also, the default `private` attribute is preferred, but `public` is also acceptable where it make sense.

- **Order of arguments.** The order of arguments to a subprogram is preferably the following: the `pass` argument, `intent(in)` arguments, `intent(in out)` arguments, `intent(out)` arguments, `optional` arguments.
- **Case.** Use lower-case everywhere except for named constants, enumerators, and preprocessor keywords. Known mathematical constants and variables can be mixed-case if more readable this way.
- **Names.** Use self-documenting variable names. The larger the scope the more informative the name should be. Do not use the Hungarian notation. Unless there is a good reason to do otherwise, the result variable of a function should be `y` with the type specified in the `function` line.
- **Spaces.** Use spaces where allowed (`in out`, `end do`, etc.). Use one space around most binary operators.
- **Kind.** Do not use syntax such as `real*8` or `real(8)`. Make use of constants such as `dp` or `kp`.
- **Line length.** Limit the line length to 80 symbols. Not only is it good for readability, but the compiler can be more informative about the line number where the error occurred. For continuation use ampersand on both lines.
- **Indentation.** 3 spaces for `do` and `if` constructs.

5.3 Version control

GADfit is versioned using the Git version control system and all contributors are expected to be fairly knowledgeable about Git. If unsure, work through the first three chapter of the Git book: <https://git-scm.com/book/en/v2> (then work through the rest of it).

Here is a summary of the rules for writing Git commit messages:

- Separate subject from body with a blank line.

- Limit the subject line to 50 characters.
- Capitalize the subject line.
- Do not end the subject line with a period.
- Use the imperative mood in the subject line.
- Wrap the body at 72 characters.
- Use the body to explain the *what* and *why* and not the *how*.

See this blog post for a full explanation: <https://chris.beams.io/posts/git-commit>.

6 Summary

Title of the program

GADfit

URL for the source code

<https://github.com/raullaasner/gadfit>

Programming languages

C++, Fortran (two independent implementations)

Distribution format

Git, gzip, zip

Nature of the problem

Nonlinear least squares regression

Method of solution

Improved Levenberg-Marquardt with multi-curve fitting

License

Apache License 2.0

Supported operating systems

Linux

7 Troubleshooting

A good place to bring up any issues is

<https://github.com/raullaasner/gadfit/issues>.

References

- [1] D. W. Marquardt, J. Soc. Indust. Appl. Math. **11**, 431 (1963).
- [2] M. K. Transtrum, B. B. Machta, and J. P. Sethna, Phys. Rev. Lett. **104**, 060201 (2010).

- [3] M. K. Transtrum, B. B. Machta, and J. P. Sethna, Phys. Rev. E **83**, 036701 (2011).
- [4] M. K. Transtrum and J. P. Sethna, arXiv:1201.5885 [physics.data-an] (2012).
- [5] H. B. Nielsen, Tech. Rep. IMM-REP-1999-05 (1999).
- [6] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, SIAM, USA, 2nd edition (2008).